ALLIANCE UNIVERSITY | E1CSA313 DESIGN PROJECT-1

Blockchain-Based Verifiable Academic Credential System

Implementation Report: Blockchain, Backend and Frontend

What was built, what each part is for, and how it works

Team	Manu M P (2411021061545) · Tarun Kumar S (2411021061552) Anilkumar U K (2411021061563) · Pruthviraj D N (L2411021061688)
Mentor	Dr. Saravanan P
Live system	https://blockchain-ecru-three.vercel.app
Source code	github.com/anilkumaruk/blockchain (branch review-2)
Date	24 October 2026

1,604	18	2,000	71
certificates on Sepolia	approved universities	students in Supabase	automated tests

Contents

#	Section	#	Section
1	What the system does	8	University wallets and trust
2	Why we use blockchain	9	Backend (technical)
3	System architecture	10	Database (technical)
4	The smart contract	11	Frontend (technical)
5	Issuing a certificate	12	Security
6	Batch issuing with Merkle trees	13	Dataset, testing and results
7	Verifying and revoking	14	Deployment, limitations and next steps

1. What the system does

Chain of Record lets universities issue degree certificates that anyone can check instantly, without phoning or emailing the university. It sits **on top of** a university's normal student records: the university keeps its data as usual, and the system adds a tamper-proof fingerprint of each certificate to a public blockchain.

There are four kinds of users:

User	What they do	Where
Platform admin	Approves or suspends universities, pauses the registry in an emergency, reads the audit log. Cannot see or issue any certificate.	/admin
University registrar	Issues certificates (one at a time or a whole class), revokes them, corrects them. Sees only their own university's certificates.	/admin
Student	Signs in with a personal email, views and downloads certificates, shares a verified profile link and QR code.	/student
Employer / verifier	Checks a certificate by number, credential ID, QR code or by uploading the PDF. No account needed.	/verify

A typical journey

- A university registers on the site; the platform admin approves it, which gives the university its own blockchain identity (wallet).
- The registrar issues a degree. The system creates the certificate PDF, computes its SHA-256 fingerprint and writes that fingerprint to the smart contract on Ethereum Sepolia.
- The student signs in and shares the certificate.
- An employer uploads the PDF or types the certificate number. The system recomputes the fingerprint and compares it with the blockchain: **valid**, **revoked**, or **invalid** (edited or never issued).

Example from the live system: during testing a new university, "achary university", registered itself on the website and was approved by the platform admin. That is the same self-registration and approval flow any real college would use.

2. Why we use blockchain

A normal database can be edited by whoever controls it, and a PDF can be edited by anyone. A blockchain is a shared, append-only record that nobody can quietly change. We use it for exactly three things:

Purpose	How the blockchain provides it
Tamper evidence	Each certificate's SHA-256 fingerprint is stored on-chain. Changing even one character of the PDF changes the fingerprint, so an edited certificate no longer matches.
Independent verification	Anyone can read the blockchain for free. An employer does not have to trust the university's website or contact the university.
Auditable history	Issuing and revoking are permanent, time-stamped events. A revoked certificate is marked revoked, never deleted, so its full history stays visible.

What goes on the blockchain, and what does not

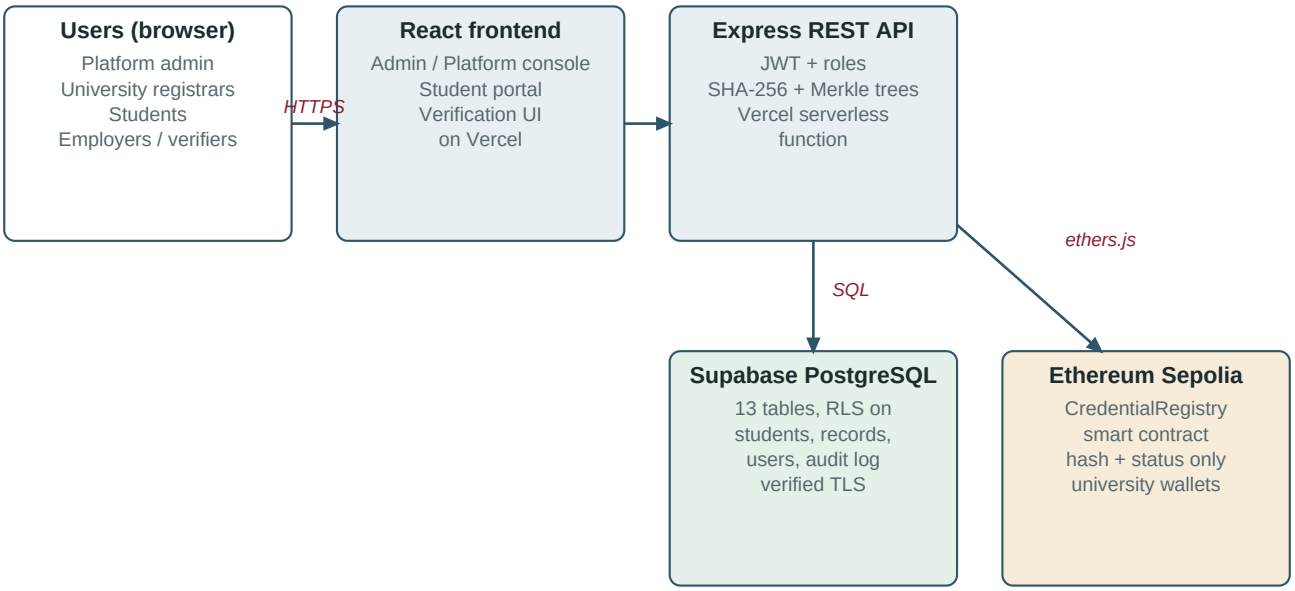
Blockchains are public and every byte costs money, so we store the minimum there and keep everything personal in the database:

Data	Stored in	Reason
SHA-256 fingerprint of the certificate	Blockchain	Proves the document is unchanged
Issuing university's wallet address	Blockchain	Proves who issued it
Issue time, revoke time, status	Blockchain	Proves it is (still) valid
Merkle root for a batch of certificates	Blockchain	One value covers a whole class
Fingerprint of the revocation reason	Blockchain	Reason can be proven later without publishing it
Student name, register number, date of birth	Database	Personal data must stay private
Grades, SGPA, enrollments, programs	Database	Detailed academic records
Passwords (hashed), audit log	Database	Accounts and accountability

Result: **no personal data is ever written to the blockchain**. Each certificate uses a fixed 128 bytes of on-chain storage, whatever the size of the PDF.

3. System architecture

The system has four parts. The browser talks only to the API; the API talks to the database and to the blockchain.



Part	Technology	Hosted on	Purpose
Frontend	React 19, Vite, Tailwind CSS	Vercel	The three portals users see
Backend API	Node.js 20, Express 4, ethers.js 6	Vercel serverless function	Business logic, security, talks to blockchain and database
Database	PostgreSQL 17	Supabase (Tokyo)	Student and certificate records, accounts, audit log
Blockchain	Solidity 0.8.24, OpenZeppelin	Ethereum Sepolia testnet	Tamper-proof fingerprints and status

Live addresses

Item	Value
Website + API	https://blockchain-ecru-three.vercel.app (API under /api)
Smart contract	0xA34BE0F943de87b2414da6Ea44A49c2bb8d5B37a on Sepolia (chain id 11155111)
Contract owner (deployer)	0xD5E2E1123700Af43EDbAa94dc1b9e30EaE8ae231
Source code	github.com/anilkumaruk/blockchain, branch review-2

4. The smart contract

The smart contract `CredentialRegistry.sol` (226 lines of Solidity) is the program that runs on the blockchain. It is built on audited **OpenZeppelin** modules: `Ownable2Step` (safe ownership transfer), `Pausable` (emergency stop) and `MerkleProof` (batch verification).

What it stores for each certificate

```
struct Credential {
    bytes32 docHash;    // SHA-256 of the certificate PDF
    address issuer;     // the university's wallet
    uint64  issuedAt;   // block time of issue
    uint64  revokedAt;  // 0 while valid
    Status  status;     // None / Valid / Revoked
}
```

Functions

Function	Who may call	What it does
issueCredential(id, hash)	Approved university	Stores one certificate fingerprint. Rejects a duplicate ID or a hash already used.
anchorBatch(root, count)	Approved university	Stores one Merkle root covering a whole batch.
revokeCredential / revokeBatchCredential	Original issuer only	Marks revoked with a time and a reason fingerprint. History is kept.
getCredential, verifyCredential, verifyBatchCredential	Anyone, free	Read-only checks used by the verifier.
setIssuer(address, name, active)	Owner (platform)	Approves or suspends a university's wallet.
pause / unpause	Owner (platform)	Emergency stop for issuing and revoking; checks still work.
transferOwnership / acceptOwnership	Owner / new owner	Two-step handover of the contract.

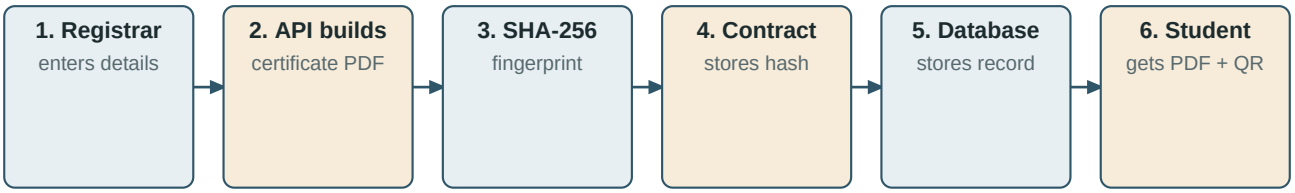
Events (the audit trail)

Every change emits an event that stays on the blockchain forever: `CredentialIssued`, `BatchAnchored`, `CredentialRevoked` and `IssuerUpdated`. The website reads these to show the on-chain history of a certificate.

Built and tested with **Hardhat**; deployed with `npm run deploy:sepolia`.

5. Issuing a certificate

When a registrar clicks *Issue Credential*, six things happen:



- **The server, not the browser, computes the fingerprint.** It rebuilds the certificate PDF from the entered fields and hashes those exact bytes, so a manipulated browser cannot register a fake hash.
- The transaction is signed with **the university's own wallet**, so the blockchain records which university issued it.
- The database row is written only after the blockchain confirms. If the blockchain is unreachable, nothing is saved and the registrar sees an error.
- If the registrar enters the student's personal email, the student can later create a login with it.

The fingerprint

SHA-256 turns any file into a 64-character code. Changing one letter gives a completely different code:

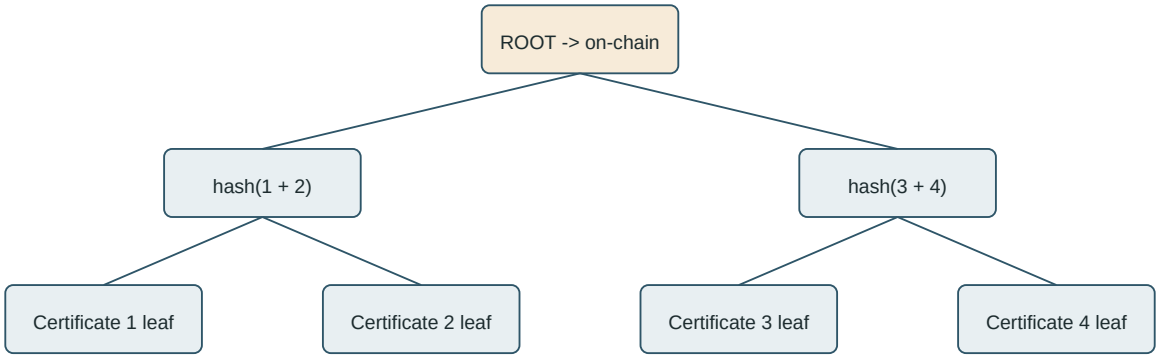
```
certificate.pdf        ->  0x6ad32c12419d7143a99d554d2b88e80d5bffa60d310719598829ea181e78a5da
same file, 1 letter    ->  (a completely different 64-character value)
```

Cost

Issuing one certificate individually uses about **125,000 gas**. Verifying is free because it only reads the blockchain. On the Sepolia test network gas costs test ETH only; the whole deployment and import used about 0.028 of the 0.05 test ETH received from a faucet.

6. Batch issuing with Merkle trees

Issuing a whole graduating class one by one would mean hundreds of transactions. Instead, the system puts all the certificates' fingerprints into a **Merkle tree** and writes only the top value, the **root**, to the blockchain in **one** transaction.



- Each certificate keeps a short **proof** (4-9 hashes) in the database that links it to the root.
- To verify, the contract recombines the certificate's fingerprint with its proof; if the result equals the stored root, the certificate is genuine.
- Leaves are hashed twice, which blocks a known attack where an internal tree node is passed off as a certificate.
- Each university's whole set of certificates was anchored this way: **17 transactions for 1,604 certificates** on Sepolia.

Cost saving (measured)

Batch size	Gas per certificate	Compared with one-by-one
1 (individual)	124,957	100%
10	12,293	9.8%
100	1,058	0.85%
500	212	0.17%

Measured on the local Hardhat network with `npm run benchmark`. On Sepolia each university batch used about 378,000 gas because of the network's own gas accounting; the saving per certificate is still over 99% for large batches.

7. Verifying and revoking

How verification works

- The verifier enters a **certificate number** (e.g. UoM/DDC/2021/001594), a credential ID, scans the **QR code**, or uploads the **PDF**.
- For a PDF, the server recomputes the SHA-256 fingerprint itself.
- The server reads the blockchain (free) and checks: does the record exist, does the fingerprint match, is the issuer an approved university, is it revoked?
- It also compares the database record with the blockchain, so if someone edited the database directly, the check fails.

Result shown	Meaning
Valid	Issued by an approved university, unchanged, not revoked
Revoked	Was genuine but the university revoked it (the reason is shown)
Invalid - hash_mismatch	The PDF was edited or never issued
Invalid - not_found	No certificate with that number
Invalid - untrusted_issuer	Issued by a wallet that is not an approved university
Invalid - record_mismatch	The database record was tampered with
Unavailable	Blockchain unreachable; the system never gives a false 'valid'

Revoking and correcting

- Revoking changes the status on the blockchain to *Revoked* with a time stamp. The original record is not deleted, so the full issue-to-revoke history stays visible.
- The reason text stays private in the database; only its fingerprint goes on-chain.
- Correcting a mistake revokes the old certificate and issues a new one, linked both ways (*replaces* / *replaced by*).
- Only the university that issued a certificate can revoke it, enforced by both the API and the contract.

8. University wallets and trust

Every approved university has its **own blockchain wallet**. This is what lets a verifier know *which* university issued a certificate, and lets the platform switch a university off.

Approval flow

- A university registers on the website and stays **pending**; it cannot sign in yet.
- The platform admin clicks *Approve*. The server creates a new wallet for the university, calls `setIssuer` on the contract to authorise it, and sends it a small amount of test ETH (0.001) to pay for its transactions.
- The university's registrar can now sign in and issue.
- If the platform admin **suspends** a university, its wallet is de-authorised on-chain: it can no longer issue, and its existing certificates show as *untrusted issuer*.

Keeping the keys safe

Each university's private key is encrypted with **AES-256-GCM** before it is stored in the database. The encryption key lives only in the server's environment settings, never in the database or the code. The platform's own key (the contract owner) is also only in the environment.

Design choice: Alliance University is an ordinary university with its own registrar, exactly like VTU or NITK. The platform admin is a separate account that belongs to no university and cannot see or issue certificates.

9. Backend (technical)

The backend is a **Node.js 20 + Express 4** REST API in `backend/server/` (about 2,100 lines). Locally it runs with `npm start`; online the same code runs as a **Vercel serverless function** (`api/index.js`).

Modules

File	Responsibility
app.js	All REST routes, input validation, verification logic, rate limits, error handling
chain.js	Blockchain bridge (ethers.js): sends transactions, waits for receipts, reads records and events
institutions.js	University approval, wallet creation, choosing the right wallet to sign with, trust checks
merkle.js	Builds Merkle trees and proofs compatible with OpenZeppelin MerkleProof
store.js	PostgreSQL access and migrations (plus an in-memory version used by tests)
auth.js / passwords.js	JWT sign-in, roles, account lockout; scrypt password hashing
wallets.js	Creates university wallets; AES-256-GCM key encryption
config.js / runtime.js / index.js	Settings and safety checks; wiring; local server start

Main API endpoints

Method and path	Who	Purpose
POST /api/auth/login	Anyone	Sign in, returns a JWT
POST /api/credentials · /batch	Registrar	Issue one / a batch (Merkle)
POST /api/credentials/:id/revoke · /reissue	Registrar	Revoke / correct
GET /api/credentials	Registrar	Own university's certificates
GET /api/verify/:id · POST /api/verify/file	Anyone	Verify by number / PDF upload
GET /api/credentials/:id/history	Anyone	On-chain event history
POST /api/auth/register/student · /api/institutions/register	Anyone	Sign-ups
/api/admin/institutions, /audit, /pause	Platform admin	Approve, audit, emergency stop

Reliability work for the public network

- Transactions on Sepolia take about 12 seconds; the bridge polls for the receipt and survives dropped connections instead of failing.
- Long jobs such as the dataset import are resumable: after any interruption they continue where they stopped.

10. Database (technical)

The off-chain data lives in **PostgreSQL 17 on Supabase**. The schema is created by four versioned SQL migrations that run automatically when the server starts.

Table	Rows (live)	Contents
institutions	18	Universities: status, wallet, type, regulator, city, AISHE code
colleges · programs	23 · 162	Colleges and degree programmes
students	2,000	Identity data (Aadhaar and mobile stored masked only)
enrollments · semester_results	2,000 · 10,407	Admission details; SGPA per semester
credentials	1,604	The on-chain-anchored record: hash, status, Merkle proof, transaction
degree_certificates	1,604	Full certificate record, 33 attributes
nad_awards	1,179	DigiLocker NAD export layout
verification_requests	666	Labelled VALID / INVALID test cases
users · audit_log	22 · 59	Accounts (hashed passwords); append-only action log

Database protection

- **Verified TLS:** the connection is encrypted and checked against Supabase's certificate.
- **Row Level Security** on all 13 tables: Supabase's public REST API cannot read anything, even with the project's public key. Only our server can.
- **Constraints:** hash format, allowed statuses and links between corrected certificates are enforced by the database itself.
- **Append-only audit log:** a database trigger blocks any edit or delete of the log.

11. Frontend (technical)

The frontend is a **React 19** single-page app built with **Vite** and styled with **Tailwind CSS**, deployed on Vercel. It talks to the API through `src/lib/api.js`, which switches between the real backend and an in-browser demo mode.

Pages

Page	Route	What it shows
Credential Center	/admin	Registrar: issue (single or CSV batch), manage, revoke, correct. Platform admin: institutions, registry pause, audit log
Student Portal	/student	Sign-in / sign-up with personal email; credentials, credential graph, verified resume, share QR
Verification	/verify	Verify by number, ID, QR scan or PDF upload; result with checks and on-chain history
Public profile	/profile/:id	A student's shareable verified profile
Register institution	/register/institution	University self-registration (pending approval)

Key components

Component	Purpose
PlatformConsole, InstitutionsPanel	Platform admin: approve / reject / suspend universities
BatchIssuePanel	Issue a whole class from a CSV in one Merkle transaction
SecurityPanel	Contract status, emergency pause, audit log
StudentAuth	Student sign-in and sign-up
VerificationResult, OnChainHistory	Verdict, checks performed, and the blockchain event trail
QrScanTab, CredentialDNA	QR scanning; visual fingerprint of a certificate's hash

The JWT from sign-in is kept for the browser session only; the browser never sees a private key or a database password.

12. Security

Risk	Protection
Edited certificate	SHA-256 on-chain; any change fails verification
Fake university	Only universities approved on-chain are trusted
Database tampering	Every check compares the database with the blockchain
Stolen passwords	scrypt hashing with a unique salt per password
Password guessing	Account locks after 5 failures; rate limit on sign-in
Forged sign-in tokens	JWT with fixed algorithm, issuer, audience and 1-hour expiry
Student claiming another's degree	Sign-up only with the email the university has on record
One university touching another's data	API checks + contract's 'original issuer only' rule
Stolen wallet key	Keys encrypted at rest; platform can pause the contract and suspend the university
Public access to the database	Row Level Security on every table
Web attacks	Security headers (Helmet), CORS allow-list, request size limits
Blockchain outage	Fails safe: shows 'unavailable', never a false pass

Secrets handling

Private keys, the database password and the encryption keys exist only in the server's environment (`backend/.env` locally, Vercel environment variables online). They are excluded from git, and the team entered the wallet key themselves; it was never typed into code or chat.

13. Dataset, testing and results

Dataset

The live system is loaded with the **Karnataka Degree Registry**, a synthetic dataset in the DigiLocker NAD layout: 17 real university names with randomly generated students and certificates (no real person's data). It contains 2,000 students, 10,407 semester results and 1,604 degree certificates: 1,344 issued, 174 provisional, and 86 not valid (revoked, withheld or cancelled), which were revoked on-chain with the dataset's reasons.

Automated tests

Test file	Tests	Covers
CredentialRegistry.test.js	8	Contract: issue, verify, revoke, permissions
batch.test.js	17	Merkle trees and proofs, batch revoke, gas saving
scenarios.test.js	11	The 10 Review-1 scenarios end-to-end + database tampering
security.test.js	18	Passwords, lockout, tokens, roles, headers, pause, config
institutions.test.js	11	University approval, wallets, isolation, student sign-up
postgres.test.js	6	Real PostgreSQL: migrations, RLS, constraints, audit log
Total	**71**	**All passing**

Measured results

Measure	Result
Genuine certificates verified as valid	50 / 50 (100%)
Certificates with one changed byte detected	50 / 50 (100%)
Revoked certificates reported as revoked	10 / 10 (100%)
Gas: issue one / revoke one / verify	~125,000 / ~45,000 / free
On-chain storage per certificate	128 bytes (fixed)

14. Deployment, limitations and next steps

How it is deployed

- Smart contract deployed to Sepolia from a dedicated test wallet (`npm run deploy:sepolia`).
- Dataset imported with `npm run import:karnataka`, which creates the universities, their wallets, anchors the certificates and loads all tables.
- Frontend and API deployed together to Vercel (`vercel --prod`); secrets set as Vercel environment variables.
- Database on Supabase; migrations applied automatically.

Known limitations

- University wallets are held by the server (encrypted). A real rollout would let each university hold its own key (hardware wallet or MetaMask) or use a key-management service.
- Student emails are trusted as entered by the university; there is no email verification link yet.
- Sepolia is a test network: it is free and public, but not permanent production infrastructure.
- Certificate files are generated from records rather than stored on IPFS.

Next steps

- Run the 666 labelled verification requests as a formal accuracy benchmark.
- OCR + NLP checking of scanned certificates (from the Review-1 literature review).
- IPFS storage for certificate files; DigiLocker / NAD integration.
- Email verification and single sign-on for students; multisig ownership of the contract.

Summary: the project now runs as a complete, deployed system: a Solidity registry on Ethereum Sepolia holding only fingerprints and status, a secured Node.js API, a private Supabase database, and three web portals, verified by 71 automated tests and a 1,604-certificate dataset.